

SyOpt: Constrained Genetic Algorithm for Near-Optimal Resilient Routing Synthesis

Csaba Györgyi¹, Kim G. Larsen², Stefan Schmid^{1,3}, and Jiří Srba^{2*}

¹ University of Vienna, Vienna, Austria

² Aalborg University, Aalborg, Denmark

³ TU Berlin, Berlin, Germany

Abstract. Modern communication networks employ local Fast Re-Routing (FRR) mechanisms that enable rapid recovery from sudden link failures in order to maintain connectivity. To prevent packet loss, re-routing must be executed immediately, relying solely on local knowledge of the affected links at each router. Configuring FRR for perfect resilience—where packets are guaranteed to be delivered as long as the source and destination remain physically connected—is a complex task. Recent advances using Binary Decision Diagrams (BDDs) provide methods to compute and efficiently store all perfectly resilient routing options. However, selecting a single resilient routing that optimizes for specific criteria, such as minimizing congestion under link failures, remains challenging and does not scale well with existing approaches. In our approach, SyOpt, we propose an efficient genetic algorithm for synthesis of near-optimal routings, constrained by a precomputed BDD representation encompassing all resilient routings, tailored to any specified objective function. Through experiments on a wide range of network topologies, we demonstrate the effectiveness of SyOpt for minimizing link congestion for routings perfectly resilient for up to one link failure. We also discuss its potential for continuous application in online monitoring and timely routing optimization, under dynamically fluctuating traffic demands.

1 Introduction

State-of-the-art communication networks rely on robust mechanisms to ensure uninterrupted data flow, even in the case of unexpected disruptions such as link failures. One such mechanism, Fast Re-Routing (FRR) [2, 5, 6, 10–12, 14–16, 19, 22–24, 34, 35, 39, 42, 46, 48, 55], provides rapid recovery by re-routing packets locally and immediately, based solely on the router’s knowledge of its neighboring links. This capability is essential to prevent packet loss and maintain network connectivity. However, configuring FRR for perfect resilience—ensuring that packets are delivered as long as a physical path exists between the source and destination—becomes increasingly complex, in particular when accounting for multiple link failures, which are common in nowadays complex networks [27, 28, 43].

* Corresponding author: srba@cs.aau.dk

Recent advances in network synthesis [29,30] have leveraged Binary Decision Diagrams (BDDs) [8,41] to compute and compactly store all perfectly resilient routing options. These advancements enable scalable representation of solution spaces, even for networks with intricate failure scenarios. However, selecting a single resilient routing that optimizes specific performance objectives, such as minimizing congestion under link failures, remains computationally challenging and existing approaches often fail to scale effectively with network size and complexity, limiting their applicability in large or dynamic systems.

Our Contributions. We address these challenges by introducing SyOpt, a novel framework that complements recent BDD-based resilience synthesis methods integrating them with a novel genetic algorithm to efficiently synthesize near-optimal routings. Our approach directly operates on the BDD representation to explore its solution space and identify routings that minimize a given objective function. By combining the compactness of BDDs with adaptive search capabilities of genetic algorithms, SyOpt offers a scalable and flexible solution to the problem of resilient routing optimization, both in static and dynamic scenarios.

We demonstrate the effectiveness of SyOpt through extensive experiments on a diverse set of real-world network topologies. Our results highlight its ability to minimize link congestion in routings that are perfectly resilient to single-link failures. Furthermore, to address the dynamic nature of modern networks, we extend SyOpt to operate in an online setting, continuously refining routing solutions in response to fluctuating traffic demands. This ensures real-time adaptability and sustained performance under varying network conditions.

While this paper focuses primarily on congestion minimization, the underlying methodology is broadly applicable to other optimization problems that rely on BDD representations of solution spaces. We believe that SyOpt represents a significant step forward in resilient network configuration and has the potential to address a wide range of challenges in communication network design and optimization as well as in other domains.

Related Work. Fast re-routing can be an order of magnitude faster than reactions in the control plane [9,26,42,49], and we refer the reader to Chiesa et al. [12] for a general overview of the field. Our work specifically contributes to fast re-routing for perfect resilience. Feigenbaum et al. [20] proved that perfect resilience is sometimes impossible, but also presented an algorithm which is always provably 1-resilient. The recent tool SyPer [29] aims provide a general and powerful BDD-based method to synthesize resilient forwarding tables. Our work complements SyPer with an efficient genetic algorithm to quickly find near-optimal routings, allowing us to take into account quantitative and performance properties as well (such as congestion), in addition to mere connectivity.

Congestion has been a key concern in traffic engineering for decades [13,25], and performance aspects recently also received attention in fast rerouting protocols [3,5,7,23,24,46]; however, these protocols do not provide any worst-case resilience guarantees in general network topologies, contrary to our approach.

Our methodology is timely and in line with several recent research efforts to render communication networks more dependable through automation [4, 17, 18, 21, 40, 47, 50, 51, 53]. For example, SyNET [17] synthesizes correct network configurations for routing protocols such as OSPF, ISIS, or BGP [12], and AalWiNes [36] provides an automated what-if verification of the policy-compliance of routes under multiple failures in MPLS networks. Protocols such as OSPF, ISIS, and BGP guarantee reconvergence after failures, however, this process is slow in the control plane; many data plane mechanisms exist, for IP-based networks [39, 48] and many others [12], and the data plane does not provide perfect resilience; hence, we do not compare to them in our simulations. AalWiNes is for verification and does not support synthesis.

2 Background and Problem Formulation

We first provide formal definitions of a network, routing function for packet forwarding and define the notion of perfect resilience, and discuss the BDD encoding of resilient routings.

2.1 Network Model

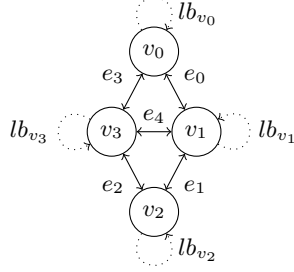
We model a network topology as a directed multigraph with loop-back edges.

Definition 1. A network is a directed multigraph $G = (V, E, r)$ where V is a finite set of nodes (routers), E is a finite set of directed edges (links), and $r : E \rightarrow \{(x, y) \mid x, y \in V\}$ is a function assigning to each edge a pair of source and target node. Furthermore, we assume that for every node $v \in V$ there is always a loop-back edge (self-loop) called lb_v such that $r(lb_v) = (v, v)$ that is used to indicate that a packet originates at a given node.

Figure 1a depicts an example of a network where all edges are bi-directional and loop-back edges (normally omitted as they are always present) are drawn using a dotted line and denote ingress interfaces for packets originating in the node.

Let us assume a fixed network $G = (V, E, r)$. A finite *path* in a network is a sequence of edges $(e_0, e_1, \dots, e_n)$ such that there are nodes $v_0, v_1, \dots, v_{n+1}$ where $r(e_i) = (v_i, v_{i+1})$ for all $0 \leq i \leq n$. The path then *connects* the node v_0 to v_{n+1} .

We assume that edges in the network can fail. A *failure scenario* $F \subseteq E$ is a subset of E and contains the set of all failed edges. In fast re-route mechanisms [12], forwarding decisions in a node v have to be resolved locally, i.e. based only on the knowledge of the set $F \cap \{e \in E \mid r(e) = (v, v'), v' \in V\}$ which contains only the failed links with v as their source node. Even though the node v has the knowledge of all failed incident links, creating a forwarding entry for every possible subset of failed links, known as the *combinatorial* routing [7], is expensive and often infeasible in practice due to high memory requirements [54]. We will hence focus on *skipping routing* [22], in the rest of the paper referred to simply as *routing*, where at every node and for every in-edge, we provide a priority list of out-edges with the semantics that the first non-failing out-edge in the

(a) Network topology where v_0 is the destination node

in-edge	node	out-edges
lb_{v_1}	v_1	e_0, e_1
e_1	v_1	e_0, e_1
e_4	v_1	e_0, e_4
lb_{v_2}	v_2	e_2, e_1
e_1	v_2	e_2, e_1
e_2	v_2	e_1, e_2
lb_{v_3}	v_3	e_3, e_2
e_2	v_3	e_3, e_2
e_4	v_3	e_3, e_4

(b) Perfectly 1-resilient routing table with objective function score of 17; the destination node is v_0

	v_0	v_1	v_2	v_3
v_0	0	0	0	0
v_1	1	0	0	0
v_2	0	0	0	0
v_3	2	0	0	0

(c) Traffic matrix

in-edge	node	out-edges
lb_{v_1}	v_1	e_0, e_4
e_1	v_1	e_0, e_1
e_4	v_1	e_0, e_4
lb_{v_2}	v_2	e_2, e_1
e_1	v_2	e_2, e_1
e_2	v_2	e_1, e_2
lb_{v_3}	v_3	e_3, e_4
e_2	v_3	e_3, e_2
e_4	v_3	e_3, e_4

(d) Perfectly 1-resilient routing table with objective function score of 13; the destination node is v_0

Fig. 1: A simple network example

list is used to forward packets arriving on the in-edge. This significantly reduces the memory requirements for storing forwarding tables.

Definition 2 (Skipping Routing). A skipping routing in a network $G = (V, E, r)$ is a partial function

$$R : E \rightarrow E^*$$

such that if $R(e) = (e_1, e_2, \dots, e_\ell)$ then $r(e) = (v', v)$ and $r(e_i) = (v, v_i)$ for some $v, v', v_1, \dots, v_\ell \in V$ where $1 \leq i \leq \ell$, i.e. the target node of e is the same as the source node of all edges e_i .

An example of a (skipping) routing for the destination node v_0 is given in Figure 1b in a form of a forwarding table. As in the example, we assume that all links are bi-directional. For simplicity, e.g. the second entry in the table means that a packet arriving on in-edge e_1 in the direction towards the node v_1 will be as the first priority forwarded on the out-edge e_0 and only if this edge is not available, we shall use the edge e_1 instead.

The general intuition is that, for a given a failure scenario F , if $R(e) = (e_1, e_2, \dots, e_\ell)$ then any packet arriving on the edge e is routed via the first available edge $e_i \notin F$ where all the edges with higher priority are failed, meaning

that $e_1, \dots, e_{i-1} \in F$. If $e_1, \dots, e_\ell \in F$ then the packet is dropped. We can now formalize the notion of a network trace for a given failure scenario.

Definition 3 (Network Trace). A path $(e_0, e_1, \dots, e_n)$ is a network trace under a routing R and a failure scenario F if for every i , $0 \leq i \leq n$, where $R(e_i) = (e'_1, e'_2, \dots, e'_\ell)$ we have $e_{i+1} = e'_j$ for some j , $1 \leq j \leq \ell$ such that $e'_1, \dots, e'_{j-1} \in F$ and $e'_j \notin F$. We say that the trace starts at node v if $r(e_0) = (v, v)$, i.e. the packet arrives to the network via the loop-back interface.

Let R be a routing and F a failure scenario. Let d be a fixed destination node for which no forwarding rules are defined. A packet starting from any node v is *delivered* to the destination d if the unique trace starting at v reaches the node d . If this is not the case, the packet can be either dropped (all out-edges in the priority list are failing) or the trace from v is infinite as we created a forwarding loop. Note that we do not define any forwarding rules for the destination node as we consider the packet delivered once it reaches d .

Returning to our running example in Figure 1a and the routing table in Figure 1b in case of no failing links ($F = \emptyset$), a packet injected at the node v_2 on the loop-back interface lb_{v_2} is forwarded via the edge e_2 to the node v_3 and as the in-edge at v_3 is e_2 , it is delivered to the destination node v_0 via the first priority out-edge e_3 . However, if the out-edge e_3 from v_3 is failing, we shall use the second priority out-edge in the routing table for the node v_3 and return the packet back to v_2 via the edge e_2 (used in the opposite direction). The routing table at node v_2 now suggests to use the edge e_1 to arrive into v_1 , following the edge e_0 that delivers the packet to the destination v_0 .

We often consider a fixed destination node. This is without loss of generality because each packet header has the information about its destination node and the routing tables for nodes with different destinations are considered as independent. Hence we can synthesize the routing tables for different destination nodes independently of each other.

A routing is perfectly k -resilient if for any failure scenario with up to k failing links, a packet gets delivered from any source to the destination node as long as these remain physically connected. For a network $G = (V, E, r)$, we denote by $G^F = (V, E \setminus F, r|_{E \setminus F})$ the network with all failed edges in F removed.

Definition 4 (Perfect Resilience). Let $d \in V$ be a destination node and k a nonnegative integer. A routing R is perfectly k -resilient, if for every node v and every failure scenario F where $|F| \leq k$, a packet starting at node v is delivered to the node d whenever there is a path connecting v to d in the graph G^F . A routing is perfectly resilient if it is perfectly k -resilient for all k .

There can exist several perfectly 1-resilient routings for a given network (see e.g. Figures 1b and 1d). The main contribution of our work is a method for selecting a resilient routing that optimizes a given quantitative criterium that e.g. minimizes congestion in the network up to a given number of failing links. In general, we can define for a given routing an objective function that summarizes e.g. the overall congestion and we are interested in a resilient routing that minimizes the objective function.

As a simple example of an objective function, let us assume that all links have the same capacity. Let $D : V \times V \rightarrow \mathbb{N}$ be a given *traffic matrix* such that $D(v, v')$ summarizes the amount of traffic from the node v to the destination v' (see Figure 1c that shows a traffic matrix with two flows, from v_1 to v_0 of size 1 and from v_3 to v_0 of size 2). Let R be a k -resilient routing. For a failure scenario F where $|F| \leq k$ and an edge $e \in E$, let $load_F(e)$ be the sum of $D(v, v')$ over all pairs of nodes v and v' such that the routing R under F delivers a packet from v to v' via a trace that uses the link e . The *objective function*

$$\max_{F \subseteq E, |F| \leq k} \sum_{e \in E} load_F^2(e) \quad (1)$$

selects the worst-case failure scenario that maximizes the squared sum of loads on all links. Consider the simple traffic matrix presented in Figure 1c where only v_1 and v_3 are sending 1 resp. 2 units of traffic to v_0 . If the network employs the routing depicted in Figure 1b, the worst-case scenario F for one-link failure (assuming that a link fails in both directions simultaneously) occurs when e_3 fails. In this case, the traffic from v_3 is routed around the network through the edges e_2 , e_1 and e_0 in order to reach v_0 , resulting in an objective function value of $2^2 + 2^2 + (2 + 1)^2 = 17$ because $load_F(e_2) = load_F(e_1) = 2$, $load_F(e_0) = 3$ and $load_F(e_4) = 0$. However, this value can be reduced to $2^2 + (2 + 1)^2 = 13$ by utilizing e_4 as a shortcut, as illustrated in Figure 1d.

The objective function in Equation (1) addresses congestion only in the worst-case scenario but does not necessarily optimize the routing in less critical failure scenarios, e.g. it will not enforce that the less critical demand of size 1 from v_1 to v_0 also uses the edge e_4 in case that e_0 is failing. In other words, the value of the objective function for the routing from Figure 1d does not change even if we in the first row for the in-edge lb_{v_1} replace the out-edges e_0, e_4 with e_0, e_1 . The objective function still evaluates to 13 even though this is a less optimal routing as it uses a longer backup path.

This limitation can be overcome by employing an alternative objective function that evaluates the worst possible load on each edge independently

$$\sum_{e \in E} \max_{F \subseteq E, |F| \leq k} load_F^2(e) . \quad (2)$$

This objective function clearly provides an upper bound compared to Equation (1) but becomes sensitive to all possible failure scenarios independently for each edge. In particular, it will favor the routing in Figure 1d over the one where the first row contains the out-edges e_0, e_1 . A further refinement of the objective function can multiply the penalty value for each failure scenario by the probability of occurrence of such a scenario, which is possible to compute e.g. under the assumption of independent link failures.

Similarly, one can incorporate a variety of other requirements in the objective function. In case of varying link capacities, the objective function can be

$$\max_{F \subseteq E, |F| \leq k} \sum_{e \in E} \left(\frac{load_F(e)}{capacity(e)} \right)^2$$

where $capacity(e)$ is the actual capacity of the link e . In all presented cases, we square the penalty per link as we try to avoid links that are almost congested and rather prefer a more uniform distribution of load over the whole network.

In fact, any custom made objective function can be used in our approach as the method is orthogonal to the definition of the objective function.

2.2 Using BDDs for Resilient Routing

Binary Decision Diagrams (BDDs), initially introduced by Lee [41] and Bryant [8], are a data structure used to represent Boolean functions. Thanks to their properties, they can effectively describe complex problems, such as resilient routings, and facilitate reasoning about the correctness of the derived solutions.

The BDD data structure represents all possible satisfying assignments of a Boolean function as a rooted directed acyclic graph (DAG) where each node is labeled by a Boolean variable and has two outgoing edges: the low (dashed) edge representing the assignment of 0 (false) to the variable at a node and the high (solid) edge that represents the assignment of 1 (true). Any BDD has two terminal nodes called 0 (false) and 1 (true). Every path from the root node reaching the node 1 represents a satisfying Boolean assignment. We shall often omit the terminal node 0 as well as all edges that lead to that node. A reduced and ordered BDD requires a fixed variable ordering on any path from the root to two terminal nodes and it reuses isomorphic substructures to achieve a more compact representation (see [8]). An example of BDD is given in Figure 5 (left) and it encodes four satisfying Boolean assignments for the variables x_1 , x_2 and x_3 , namely 110, 111, 101 and 010. Observe that on the left-most path the variable x_3 is missing, meaning that if x_1 and x_2 are set to truth, any value of x_3 makes the assignment satisfying. The BDD data structure supports not only basic logical operators but also universal and existential quantifiers, all of which can be performed in polynomial time relative to the size of the input BDDs.

Boolean functions, represented as BDDs, can be utilized to encode complex relationships. A finite set S can be represented using $n = \lceil \log(|S|) \rceil$ Boolean variables: $\bar{x} = (x_0, x_1, \dots, x_n)$. After fixing an ordering of the elements $s_0, s_1, \dots, s_{|S|-1} \in S$, a truth assignment μ to $\bar{x}$ corresponds to the binary representation of a natural number $n(\mu) \in \mathbb{N}$, encoding the $n(\mu)$ -th element of S .

SyPer [29] formulated the problem of representing all perfectly k -resilient routings using Boolean functions and storing them in the BDD data structure. After defining valid transitions of a packet inside a graph based on the routing rules, it iteratively computes the function's reflexive transitive closure using a fixed-point calculation to encode possible delivery scenarios. Finally, it derives a Boolean function through quantification, representing *all* possible resilient routings. In this paper, we suggest an improved version of SyPer, where undirected edges are represented using two directed edges, resulting in more favorable memory footprints and runtimes. However, these enhancements are beyond the scope of this paper and do not affect our scientific contribution that

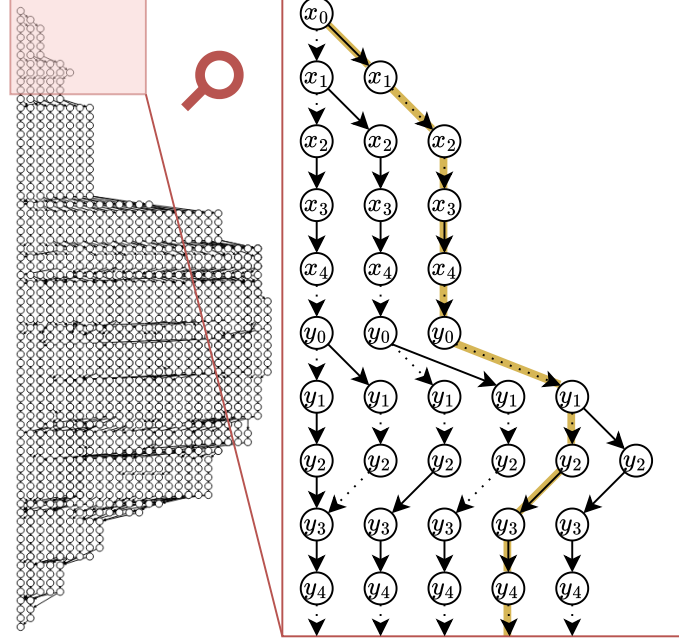


Fig. 2: BDD representation of all perfectly 1-resilient routings for the network from Figure 1a with the destination v_0 (there are 9388 paths representing all possible 1-resilient routings). The highlighted path prefix encodes the first row of the routing table from Figure 1b.

focuses on selecting an optimal routing from a given BDD representation of all resilient routings.

BDD Encoding Example. Consider the network depicted in Figure 1a, which offers numerous possible routing options. Leveraging BDDs, we can efficiently compute and represent all possible perfectly 1-resilient routings. Figure 2 illustrates a BDD corresponding to all 9388 possible 1-resilient routings for delivering packets to v_0 . The enlarged first 10 levels of the BDD encode two values, $\bar{x}$ and $\bar{y}$, using the method described above. Here, $\bar{x}$ and $\bar{y}$ are each vectors of five Boolean variables and represent the possible preference lists for a packet arriving on the loop-back interface lb_{v_1} at node v_1 . An example of a perfectly 1-resilient routing shown in Figure 1b corresponds to the highlighted path in the magnified BDD part. In this assignment (starting with the least significant bit) the variables $\bar{x}$ take the value 9 (01001 in binary) and the $\bar{y}$ variables take the value 12 (01100 in binary). These two numbers represent the edges e_0 and e_1 , respectively (in some fixed ordering of edges). Similarly, the routing table from Figure 1d is represented by the same initial path on $\bar{x}$ variables but the $\bar{y}$ variables encode the value 14 (01110 in binary) which is the right-most path in the BDD and corresponds to the edge e_4 . Hence the out-edges in the first line

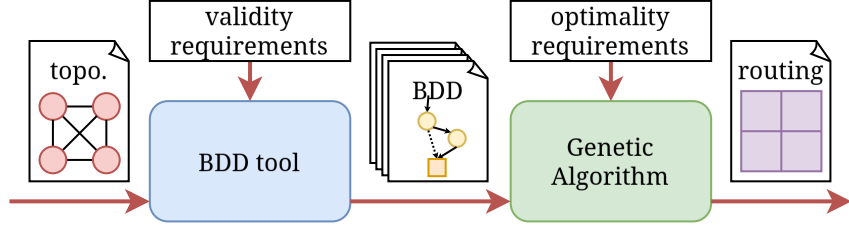


Fig. 3: SyOpt system overview

of the table are e_0 and e_4 . The resulting BDD encodes the whole set of 9388 perfectly 1-resilient routing tables and contains a total of 1515 nodes with only 26 nodes at its widest level, showcasing the compressing ability of BDDs.

Optimization Challenges. As demonstrated by SyPer [29] and SyRep [30], BDDs provide an efficient and compact way to represent all possible perfectly k -resilient routings. However, it is challenging to identify, usually among a large number of perfectly resilient solutions, the one that is optimal with respect to some given optimization criteria. In our use case, the different perfectly 1-resilient routings can provide different link utilizations, in particular in case of link failures. Hence deploying a random routing from the solution space on one hand guarantees perfect resilience, however, it can be causing congestion either in the basic data plane or in some of the possible failure scenarios.

The main contribution of this paper is a method based on genetic programming that allows us to efficiently select a near-optimal solution from the large solutions space stored in a BDD. We decided not to encode the additional performance criteria directly into BDD during their construction because (i) BDD computations can be resource-intensive and incorporating additional requirements may lead to unacceptable memory usage and CPU time, and (ii) resilient routing calculations are typically performed for each destination individually, whereas performance metrics often require analyzing all possible destinations simultaneously, significantly increasing BDD size.

3 Genetic Algorithm using BDDs

Genetic algorithms (GAs) [33, 37, 52] are a well-known probabilistic approach that mimics the evolutionary process of biological organisms. To ensure the selection of resilient routings, the standard GA must be adapted for use with BDDs.

Main Building Blocks. A genetic algorithm typically begins with a randomly generated *population*, where each *individual* is represented by a *genome*, often in the form of a bit sequence. The individuals are then evaluated using an *objective function*, and the highest-performing ones are generally retained for the next generation. The remaining positions are filled through a breeding process, where the genomes of two individuals are combined (*crossover*) and may

Algorithm 1 Genetic BDD-Constrained Algorithm

```

1: population  $\leftarrow$  randomly select POPULATION_SIZE individuals from the given
   BDD // see Section 3.2
2: for  $i \leftarrow 2$  to NUM_OF_GENERATIONS do
3:   survivors  $\leftarrow$  select KEEP_BEST best performing individuals, based on the given
   OBJECTIVE_FUNCTION
4:   descendants  $\leftarrow \emptyset$ 
5:   for  $j \leftarrow 1$  to POPULATION_SIZE - KEEP_BEST do
6:     tournament  $\leftarrow$  randomly select TOURNAMENT_SIZE individuals from the
     current population
7:     parent1, parent2  $\leftarrow$  two best individuals from the tournament, based on the
     OBJECTIVE_FUNCTION
8:     child  $\leftarrow$  do a BDD-constrained crossover with parent1 and parent2
     // see Section 3.3
9:     child  $\leftarrow$  mutate child with probability MUTATION_PROBABILITY
     // see Section 3.4
10:    descendants  $\leftarrow$  descendants  $\cup$  { child }
11:   end for
12:   population  $\leftarrow$  survivors  $\cup$  descendants
13: end for
14: return the best performing individual from the current population based on the
   OBJECTIVE_FUNCTION

```

undergo *mutation* to create a new genome. The selection process for breeding allows for a variety of approaches. In this paper, we employ *tournament selection* [45], where n genomes are randomly selected (repetition is allowed) from the current population, and two highest-performing individuals are chosen for breeding, making the selection more immune to noise. This process is repeated until the next generation is populated. Tournament selection favors the best performers while still giving each individual a chance to participate. A high-level pseudocode representation, adapted to our BDD setting, can be found in Algorithm 1.

Challenges. The challenges begin with the random selection of the initial population. How can we randomly select individuals (routings) from a BDD in such a way that each individual has an equal probability of being chosen? Second, how can we implement crossover while ensuring that the resulting genome represents a valid perfectly resilient routing? Lastly, how should mutation be introduced to allow limited changes to the genome while preserving perfect resilience? In what follows, we propose solutions to each of these challenges.

3.1 System Overview

Figure 3 provides an overview of our proposed solution, SyOpt, which consists of two primary components: *BDD tool* and the *genetic algorithm*. The BDD tool is responsible for representing all possible solutions using BDDs. It takes the network topology as input and generates all valid solutions in the form of one or

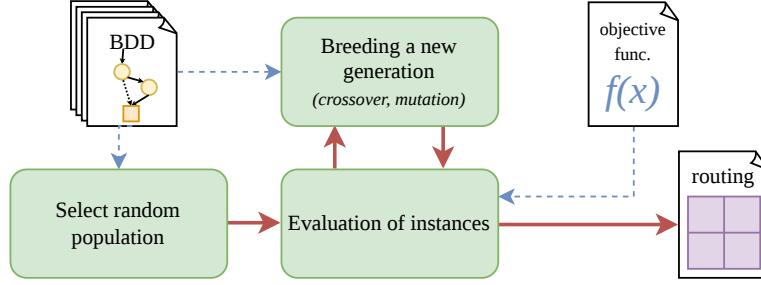


Fig. 4: Components of the genetic algorithm

more BDDs, based on the specified *validity requirements* that guarantee perfect k -resilience. In our specific use case, we utilized an enhanced version of SyPer [29] to generate for each destination all possible resilient routings and represented them as a BDD.

In the subsequent step, the genetic algorithm depicted in Figure 4, processes the BDDs to identify an optimized solution based on the provided optimality requirements, represented by an *objective function*. We propose that the genome consists of the sequence of Boolean decisions (1 represents a high edge and 0 a low edge) made while traversing a BDD. However, it is crucial to ensure that the standard operations of the genetic algorithm do not violate the *validity requirements* while efficiently exploring the solution space. To achieve this, the random selection of instances, as well as crossover and mutation operations, must be adapted to be consistent with the BDD, i.e. every individual must form a valid path from the root node of the BDD to its terminal node 1. In the final step, the Genetic Algorithm produces a solution that satisfies both the validity requirements and is optimized according to the specified optimality criteria. In our use case, this results in a resilient routing with favorable link utilization characteristics.

3.2 Random Sampling

Sampling a random satisfying assignment for a given BDD may seem as a straightforward task by performing a walk from the root variable of the BDD until the terminal node 1 and whenever both the low and high edge exist, selecting one of them with uniform probability. However, this procedure does not guarantee that each individual (assignment) is selected with equal probability as demonstrated on our BDD example from Figure 5 that contains four satisfying assignments and hence each of them should be selected with probability 0.25. The assignment of equal probability to low and high edges, as depicted on the left of Figure 5, results in the fact that in half of the cases we select the assignment 010 as the right-most path representing this assignment is selected with probability $0.5 \cdot 1 \cdot 1 = 0.5$.

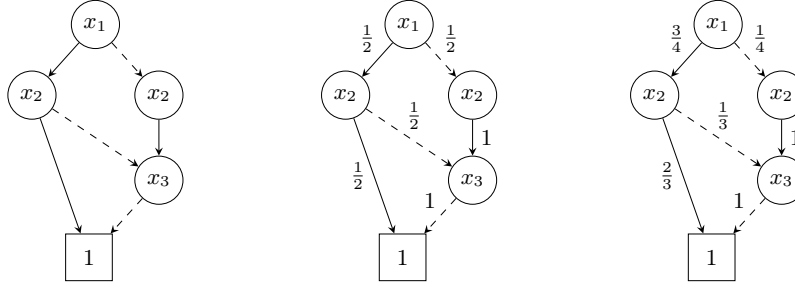


Fig. 5: BDD with the satisfying assignments: 110, 111, 100, 010 (on the left) annotated with probabilities for naive (in the middle) and improved (on the right) random BDD sampling.

Formal Requirement. We instead aim to ensure that from a given set S of satisfying assignments over the variables $x_1, \dots, x_n$, each element is selected with equal probability using the BDD. Each $s = s_0 \dots s_n \in S$ represents a sequence of Boolean variable assignments (where the variable x_i is assigned the value s_i), corresponding to the decisions made within the BDD to reach the terminal node 1. Furthermore, let P_s denote the probability of picking the $s \in S$ satisfying assignment. Based on this notation, our formal requirement is as follows:

$$\forall s \in S : P_s = 1/|S| .$$

In order to achieve this goal, for each node v in the BDD, we have to determine appropriate $P_{v,0}$ and $P_{v,1}$ values, representing the probability of choosing *low* or *high* edge at node v , respectively. Naturally, these probabilities must satisfy $P_{v,0} + P_{v,1} = 1$. We can now rewrite our formal requirement so that for every assignment $s = s_0 s_1 \dots s_n \in S$ of the variables $x_1, \dots, x_n$ that visits the BDD nodes $v_1, \dots, v_n$ such that v_i is labeled by the variable x_i (if for some variable x_i there is no corresponding BDD node, we assume that v_i is a dummy node d with the probabilities $P_{d,0} = P_{d,1} = 0.5$), we must satisfy

$$P_{v_0, s_0} \cdot P_{v_1, s_1} \cdot \dots \cdot P_{v_n, s_n} = 1/|S| . \quad (3)$$

Solution. For every node v in the BDD, we assign the probabilities $P_{v,0}$ and $P_{v,1}$ as follows. Let S_v , where v is labeled with variable x_i , denote the set of all satisfying assignments over the variables $x_i, \dots, x_n$ while assuming all paths from v to the terminal node 1. Let v_{low} and v_{high} be the nodes that are reached from v via the low and high edge, respectively. We define $P_{v,0} = |S_{v_{low}}|/|S_v|$ and $P_{v,1} = |S_{v_{high}}|/|S_v|$. Intuitively, the probability of taking the low or high edge in a node v is proportional to the number of solutions that can be achieved from the low and high child. The computed probabilities for our running example are depicted in Figure 5 on the right. We can now check that the probability of selecting the assignment 100 is $\frac{3}{4} \cdot \frac{1}{3} \cdot 1 = 0.25$, and similarly the probability of selecting e.g. 110 is also 0.25 as $\frac{3}{4} \cdot \frac{2}{3} \cdot \frac{1}{2} = 0.25$ (because the node for x_3 is

missing, we assume a dummy node with probability of the low edge 0.5). The following lemma can be proven by substitution.

Lemma 1. *The assigned probabilities $P_{v,0}$ and $P_{v,1}$ satisfy Equation (3), implying that any satisfying assignment $s \in S$ is selected with the uniform probability $1/|S|$.*

Implementation Considerations. The number of satisfying assignments from a given node can be efficiently computed using a bottom-up approach via dynamic programming. This technique operates with a linear runtime complexity relative to the number of nodes in the BDD.

3.3 Crossover

Crossover is the process of combining two genomes to produce a new instance. While a path in the BDD can be represented as a sequence of binary decisions, we can not simply split and merge these sequences as is typically done in genetic algorithms. Doing so would likely result in a genome that describes an invalid path within the BDD, thus the resulting routing may no longer be resilient.

General Method. Crossover of two BDD paths must take the structure of the BDD into account. Our proposed method randomly selects a BDD node v where the two input paths intersect. The new path is constructed by taking the first path up to node v , and continuing from v along the second path. A simple example is depicted on Figure 6a where x_3 acts as the intersection of the two parents. This approach guarantees that the resulting genome always represents a valid path within the BDD. However, the method has a corner case: if the two paths never intersect, crossover is not possible. In such cases, one of the two BDD paths is randomly selected.

Use Case-Specific Observations. The method ensures that the newly generated genome always represents a resilient routing, potentially improving link-utilization. In our specific use case, where each genome consists of multiple BDD paths (one for each destination), the impact of the non-crossing corner cases is further minimized. In the worst-case scenario, where none of the BDD paths intersect, the new genome simply inherits entire BDD paths from its parents.

3.4 Mutation

The goal of mutation is to introduce minor variations in the genome, potentially enhancing its fitness. A key advantage of mutation is that it can generate genomes that would be unattainable through crossover alone. However, as with crossover, randomly flipping bits in the genome can easily result in invalid paths. In our specific use case, this could lead to invalid or, at the very least, non-resilient routing.

General Method. The input to the mutation process is a BDD path. First, we randomly select a BDD node v along the path that has two outgoing edges. The new path follows the original path up to node v . At node v , we take the

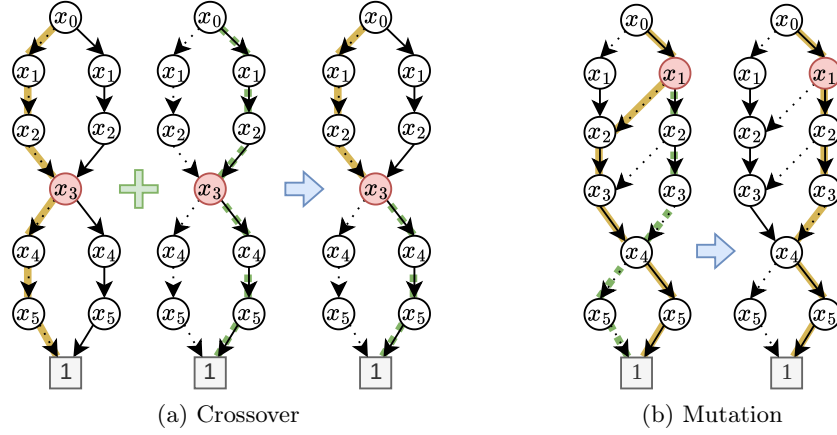


Fig. 6: BDD-constrained genetic algorithm operators

alternate edge leading to node v' , thereby ensuring a change in the genome. From node v' , a random path is selected to the terminal node of the BDD, following the method outlined in Section 3.2. Additionally, we aim to minimize the extent of changes introduced by the mutation. If the random path first intersects the original path at some node v'' , the new individual inherits the remainder of the original path from node v'' onward. An example is depicted in Figure 6b where the mutation point v is in x_1 , with the random tail merged back in x_4 acting as v'' . This adjustment is essential; without it, if v is the first node, we would essentially create a new random individual rather than introducing a targeted modification. It is important to note that mutation is not possible for paths that do not contain a node v with two outgoing edges. In such cases, the path remains unchanged. However, this can only occur when the BDD represents only a single satisfying assignment, rendering every optimization trivial.

Use Case-Specific Observations. This method allows for making limited modifications to an existing routing while still ensuring resilience. The general approach alters the path at only one point but can be repeated multiple times to increase the extent of changes introduced.

3.5 Objective Functions

The objective function is responsible for providing a metric that can be minimized through the genetic algorithm.

Flexibility. Our design ensures that the strict requirement of resilience is consistently maintained, while the objective function allows for considerable flexibility. Although we currently optimize for the squared sum of maximum link utilizations, the formula can be easily adapted to optimize for different load characteristics. Furthermore, it is straightforward to define entirely new metrics, such as the maximum path length, as specifying the objective function does not require any specialized knowledge of BDDs.

Dealing with Changes. Certain objective functions, such as link utilization, may fluctuate over time. We propose that the final generation from a previous optimization run can be reused for a new objective function, provided the change is not too drastic. In cases where rapid, significant changes occur, it is possible to maintain a set of pre-trained populations for common objective functions (e.g., typical traffic matrices). In such scenarios, the algorithm can begin with the population corresponding to the most similar objective function, such as the closest traffic matrix. It is also important to note that the evaluation of objective functions is generally well-suited to parallel and distributed processing. This enables significant potential for speedup, particularly in situations that require quick adaptation. Nevertheless, in our experiments presented in the next section, we restrict ourselves to single-core CPU experiments which provides already a sufficient capacity.

4 Evaluation and Applicability

The proposed genetic algorithm is implemented in Python to facilitate the evaluation of the method. Similarly to SyPer [31], we use the Omega [1] library to generate the BDDs using our improved representation. Reproducibility package is available at [32].

It is important to note that the generation of BDDs and the execution of the genetic algorithm (GA) to search for the optimal solution are distinct steps. As this paper focuses solely on the evaluation of the GA, we pre-generated the BDDs that encode all possible perfectly 1-resilient routings for each destination node. Using a server with 400 GB of RAM and a 4-hour timeout, we successfully generated the BDDs for 83 networks from the Internet Topology-Zoo [38]. The topology sizes where the BDD generation succeeded range from a few nodes to up to 60 nodes.

The GA solution was evaluated on an HPC cluster, with each experiment executed on a single core. It is worth noting that our implementation does not utilize parallel computations that have a significant potential to further reduce the computation times, given the parallelizable nature of genetic algorithms and their associated objective functions. Such speedups achieved using parallelization are left for the future work.

We have performed multiple experiments for different parameter settings of our genetic algorithm and in the rest of this section, we fix mutation probability to 0.2, population size to 30, number of instances kept without modifications to 10 and tournament size to 5, as these were the best performing parameters providing a good compromise between the speed and the learning capabilities. We evaluate the objective function from Equation (2) on failure scenarios with single bi-directional link failures (assuming that if a link fails in one direction then it is down also in the opposite direction).

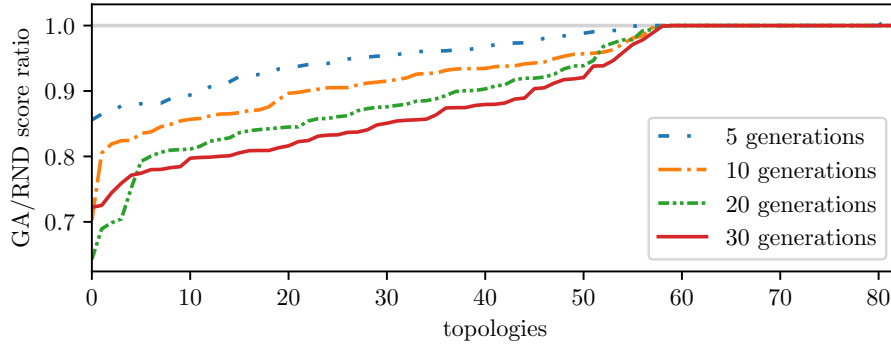


Fig. 7: Score ratios of genetic vs. random selection

4.1 Genetic Algorithm Performance

We shall start by evaluating the performance of the genetic algorithm (GA) compared to the random selection algorithm (RND) that randomly chooses a routing from the BDD representation of all resilient routings. Figure 7 compares the influence of the number of generations (from 5 to 30) with the RND algorithm where we plot the ordered ratios of the objective function of the best individual identified by GA divided by the value of the objective function of the best randomly selected routing. We always generate as many random routings as is the number of individuals produced by the genetic algorithm (610 individuals in our experimental setup). As Figure 7 shows, our GA consistently finds routings with better scores than RND. Also, increasing the number of generations clearly contributes to finding routings with better scores. In the best cases, the score is better by up to 25% but it provides clear improvements for most of the considered network topologies, except for the last 25 topologies where the scores of the routings found by GA and RND are equal. This is caused by the fact that these topologies are essentially trees (contain fewer number of edges than nodes) and hence the routings are uniquely determined and there is nothing to optimize. See Figure 8 that shows how the ratios for scores of the 30-generations GA divided by RND depend on the ratio of the number of edges divided by the number of nodes in each considered topology. This demonstrates that improvements are possible only on networks that have more edges than nodes.

We conclude that our genetic algorithm finds routings with better scores compared to a random selection of equally many routings as the number of individuals considered in GA (and hence using about the same CPU time as evaluating the score function of an individual is the most expensive operation). At the same time, increasing the number of generations clearly helps improving the performance of our genetic algorithm.

Moreover, selecting a small set of random routings is unlikely to find a reasonable routing with a good score.

On the other hand, Figure 9 shows the convergence rate towards the optimal solution in our eight selected topologies as we increase the number of generations

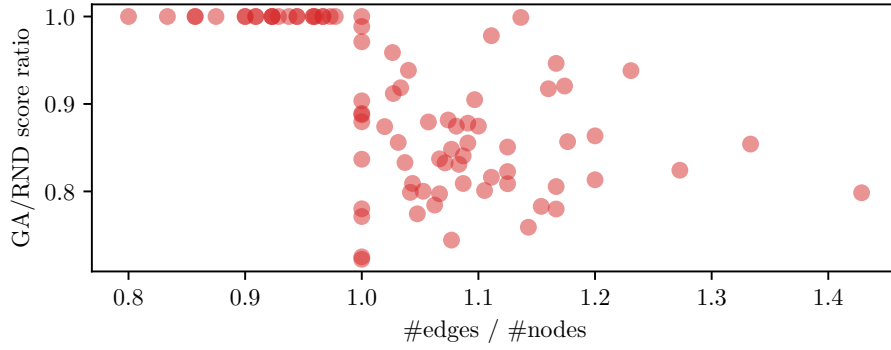


Fig. 8: Distribution of score ratios for 30 generations GA

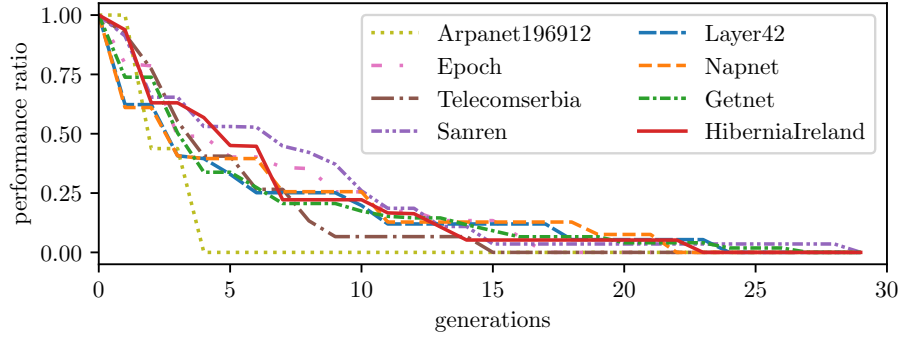


Fig. 9: Performance ratios per number of generations

on the x-axis. The y-axis depicts the values, produced by the score function, normalized into the interval $[0,1]$ where 1 means the worst score and 0 is the best achieved score. We can observe that e.g. for Arpanet196912 our genetic algorithm finds the best score in just 4 generations, while for others like Sanren it requires up to 30 generations.

Finally, in Figure 10 we present (for all considered network topologies) the expected time requirements of our genetic algorithm. The x-axis represents the size of the topology (number of nodes plus the number of edges) and the y-axis shows the average CPU time needed to compute one complete generation of individuals in our genetic algorithm. We can observe that the CPU time per generation typically increases with the topology size but there are also a few smaller topologies with high CPU time requirements. The plot shows that the worst-time per generation is around 140 seconds, which makes the application of the method in an online setting unfeasible as e.g. computing 10 to 30 generations (required for achieving the best scores) can take between 20 minutes up to one hour. Assuming that network traffic changes typically more frequently, we need to develop a solution that allows us to react in the matter of minutes and not hours. Such an online genetic algorithm is discussed in the next subsection.

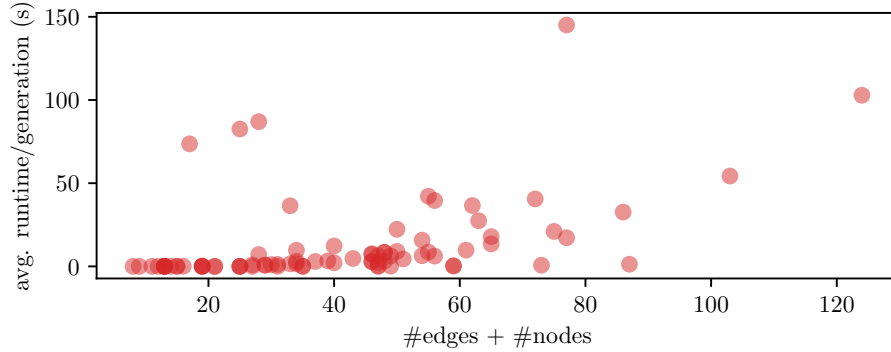


Fig. 10: Average time per generation vs. topology size

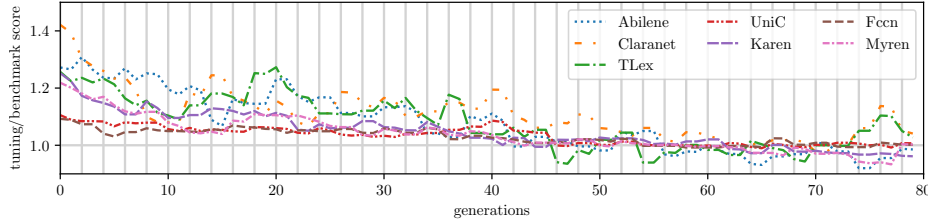


Fig. 11: Online genetic algorithm compared to the offline benchmark score run for 30 generations

4.2 Dealing with Fluctuating Traffic Loads

As network traffic is dynamically changing over time, we shall now propose an adaptation of our genetic algorithm in order to deal with frequently changing traffic matrices. As discussed in the previous subsection, it is impractical to recompute an optimal routing after every change in the traffic matrix as such process can take up to one hour and applying the computed routing after such a delay does not necessarily address the current traffic demands anymore.

In our proposed online solution, we shall instead continuously run the genetic algorithm along with the real traffic and after a fixed number of computed generations, we shall repeatedly update the current objective function according to the measured traffic load, however, keeping the current generation of individuals. Whenever the traffic changes reach a given threshold or after a predefined number of generations, we can select the currently best individual (according to the objective function based on the most recent traffic matrix) and apply such routing to be prepared for link outages.

The assumption here is that traffic changes happen more or less continuously and follow given patterns where certain demands reach peak traffic during the day and other demands during the night (for example backup and maintenance tasks). Such a fluctuation of internet traffic can be modeled, for example, using sinusoids [44, 56].

The simulation with randomly generated sinusoid traffic patterns now assumes that after every two generations computed by our algorithm, we update the current traffic matrix according to the current demands for each flow and this change is reflected in an updated objective function. Figure 11 demonstrates the development of the quality of the best computed routing over time (number of computed generations). By quality we consider the value of the current score divided by the score of statically running the genetic algorithm for 30 generations for the current but fixed traffic matrix). We can observe in the plot that over time (in about 40–50 generations) all networks reach a quality of the online computed solution that is comparable (and sometimes even better, with some minor fluctuations) than the one computed statically for 30 generations. This demonstrates that our online genetic algorithm will over time match the performance of the offline approach while giving us an immediate access to the currently best possible routing available in the population.

5 Conclusion

We introduced SyOpt, a novel method and tool for synthesizing near-optimal, perfectly resilient routings that minimize congestion under link failure scenarios. SyOpt leverages recent advancements in compactly encoding all perfectly resilient routings using Binary Decision Diagrams (BDDs). This representation allows us to efficiently manage a potentially vast number of routing options. To identify an optimal routing that minimizes a specific objective function, we proposed a novel genetic algorithm that directly operates on BDD assignments.

We demonstrated the effectiveness of this approach by applying it to the problem of congestion minimization in the presence of link failures. Experiments on real-world network topologies confirmed that our genetic algorithm significantly outperforms blind selection. Additionally, to address the demands of dynamic network environments, the online version of the solution adapts to fluctuating traffic patterns.

Importantly, while we focused on congestion minimization, the versatility of our approach extends to other domains that utilize BDD representations for solution spaces. Future work should explore SyOpt’s applicability to a wider range of problems, exercising its flexibility and strength.

Acknowledgements. The paper was supported by Villum Investigator Grant S4OS, 2020–2027 and the German Research Foundation (DFG), SPP 2378 framework, grant 511099228 (ReNO-2), 2025-2029.

References

1. Omega Python package (2023), <https://github.com/tulip-control/omega>
2. Bankhamer, G., Elsässer, R., Schmid, S.: Local fast rerouting with low congestion: A randomized approach. In: ICNP (2019)

3. Bankhamer, G., Elsässer, R., Schmid, S.: Randomized Local Fast Rerouting for Datacenter Networks with Almost Optimal Congestion. In: 35th International Symposium on Distributed Computing (DISC). pp. 9:1–9:19 (2021)
4. Beurer-Kellner, L., Vechev, M., Vanbever, L., Veličković, P.: Learning to configure computer networks with neural algorithmic reasoning. *Advances in Neural Information Processing Systems* **35**, 730–742 (2022)
5. Borokhovich, M., Pignolet, Y.A., Schmid, S., Tredan, G.: Load-optimal local fast rerouting for dense networks. *IEEE/ACM Transactions on Networking* **26**(6), 2583–2597 (2018)
6. Borokhovich, M., Schiff, L., Schmid, S.: Provable data plane connectivity with local fast failover: introducing openflow graph algorithms. In: HotSDN. pp. 121–126. ACM (2014)
7. Borokhovich, M., Schmid, S.: How (not) to shoot in your foot with SDN local fast failover - A load-connectivity tradeoff. In: OPODIS. pp. 68–82 (2013)
8. Bryant: Graph-based algorithms for boolean function manipulation. *IEEE Transactions on Computers* **C-35**(8), 677–691 (1986). <https://doi.org/10.1109/TC.1986.1676819>
9. Busch, C., Surapaneni, S., Tirthapura, S.: Analysis of link reversal routing algorithms for mobile ad hoc networks. In: ACM SPAA. p. 210–219 (2003)
10. Canini, M., Kuznetsov, P., Levin, D., Schmid, S.: A distributed and robust SDN control plane for transactional network updates. In: INFOCOM. pp. 190–198. IEEE (2015)
11. Chiesa, M., Gurtov, A.V., Madry, A., Mitrovic, S., Nikolaevskiy, I., Schapira, M., Shenker, S.: On the resiliency of randomized routing against multiple edge failures. In: ICALP. pp. 134:1–134:15 (2016)
12. Chiesa, M., Kamiński, A., Rak, J., Rétvári, G., Schmid, S.: A survey of fast-recovery mechanisms in packet-switched networks. *IEEE Communications Surveys & Tutorials* **23**(2), 1253–1301 (2021)
13. Chiesa, M., Kindler, G., Schapira, M.: Traffic engineering with equal-cost-multipath: An algorithmic perspective. *IEEE/ACM Transactions on Networking* **25**(2), 779–792 (2016)
14. Chiesa, M., Nikolaevskiy, I., Mitrovic, S., Panda, A., Gurtov, A.V., Madry, A., Schapira, M., Shenker, S.: The quest for resilient (static) forwarding tables. In: INFOCOM. pp. 1–9 (2016)
15. Chiesa, M., Nikolaevskiy, I., Panda, A., Gurtov, A.V., Schapira, M., Shenker, S.: Exploring the limits of static failover routing (v4). arXiv technical report **abs/1409.0034.v4** (2016)
16. Dai, W., Foerster, K.T., Schmid, S.: A tight characterization of fast failover routing: Resiliency to two link failures is possible. In: Proc. ACM SPAA. p. 153–163 (2023)
17. El-Hassany, A., Tsankov, P., Vanbever, L., Vechev, M.: Network-wide configuration synthesis. In: CAV’17. pp. 261–281. Springer (2017)
18. El-Hassany, A., Tsankov, P., Vanbever, L., Vechev, M.: Netcomplete: Practical network-wide configuration synthesis with autocompletion. In: USENIX NSDI. pp. 579–594 (2018)
19. Feigenbaum, J., Godfrey, B., Panda, A., Schapira, M., Shenker, S., Singla, A.: Brief announcement: On the resilience of routing tables. In: Proc. ACM PODC. p. 237–238 (2012)
20. Feigenbaum, J., Godfrey, B., Panda, A., Schapira, M., Shenker, S., Singla, A.: Brief announcement: on the resilience of routing tables. In: PODC. pp. 237–238. ACM (2012)

21. Finkbeiner, B., Giesekeing, M., Hecking-Harbusch, J., Olderog, E.R.: Model checking data flows in concurrent network updates (full version). arXiv preprint arXiv:1907.11061 (2019)
22. Foerster, K.T., Hirvonen, J., Pignolet, Y.A., Schmid, S., Trédan, G.: On the feasibility of perfect resilience with local fast failover. In: Proc. SIAM APOCS. pp. 55–69 (2021)
23. Foerster, K.T., Pignolet, Y.A., Schmid, S., Trédan, G.: Local fast failover routing with low stretch. *Computer Communication Review* **48**(1), 35–41 (2018)
24. Foerster, K.T., Pignolet, Y.A., Schmid, S., Trédan, G.: CASA: congestion and stretch aware static fast rerouting. In: INFOCOM. pp. 469–477. IEEE (2019)
25. Fortz, B., Thorup, M.: Internet traffic engineering by optimizing ospf weights. In: Proceedings IEEE INFOCOM 2000. conference on computer communications. Nineteenth annual joint conference of the IEEE computer and communications societies (Cat. No. 00CH37064). vol. 2, pp. 519–528. IEEE (2000)
26. Gafni, E., Bertsekas, D.P.: Distributed algorithms for generating loop-free routes in networks with frequently changing topology. *IEEE Trans. Communications* **29**(1), 11–18 (1981)
27. Gill, P., Jain, N., Nagappan, N.: Understanding network failures in data centers: measurement, analysis, and implications. In: SIGCOMM. pp. 350–361. ACM (2011)
28. Govindan, R., Minei, I., Kallahalla, M., Koley, B., Vahdat, A.: Evolve or die: High-availability design principles drawn from googles network infrastructure. In: SIGCOMM. p. 58–72 (2016)
29. Gyorgyi, C., Larsen, K., Schmid, S., Srba, J.: SyPer: Synthesis of perfectly resilient local fast re-routing rules for highly dependable networks. In: IEEE INFOCOM’24. pp. 1–10. IEEE (2024), to appear.
30. Gyorgyi, C., Larsen, K., Schmid, S., Srba, J.: SyRep: Efficient synthesis and repair of fast re-route forwarding tables for resilient networks. In: Proceedings of the 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN’24). pp. 483–494. IEEE (2024)
31. Gyorgyi, C., Larsen, K.G., Schmid, S., Srba, J.: Syper: Synthesis of perfectly resilient local fast rerouting rules for highly dependable networks. In: IEEE INFOCOM (2024)
32. Györgyi, C., Larsen, K.G., Schmid, S., Srba, J.: Reproducibility package for “syopt: Constrained genetic algorithm for near-optimal resilient routing synthesis” (Sep 2026). <https://doi.org/10.5281/zenodo.22705631>, <https://doi.org/10.5281/zenodo.22705631>
33. Holland John, H.: *Adaptation in nature and artificial systems* (1975)
34. Holterbach, T., Molero, E.C., Apostolaki, M., Dainotti, A., Vissicchio, S., Vanbever, L.: Blink: Fast connectivity recovery entirely in the data plane. In: USENIX NSDI. pp. 161–176 (2019)
35. Jensen, J.S., Krøgh, T.B., Madsen, J.S., Schmid, S., Srba, J., Thorgersen, M.T.: P-Rex: Fast verification of MPLS networks with multiple link failures. In: CoNEXT. pp. 217–227. ACM (2018)
36. Jensen, P.G., Kristiansen, D., Schmid, S., Schou, M.K., Schrenk, B.C., Srba, J.: AalWiNes: A fast and quantitative what-if analysis tool for mpls networks. In: ACM CoNEXT. p. 474–481 (2020)
37. Katoch, S., Chauhan, S.S., Kumar, V.: A review on genetic algorithm: past, present, and future. *Multimedia tools and applications* **80**, 8091–8126 (2021)

38. Knight, S., Nguyen, H.X., Falkner, N., Bowden, R., Roughan, M.: The internet topology zoo. *IEEE Journal on Selected Areas in Communications* **29**(9), 1765–1775 (2011)
39. Kwong, K.W., Gao, L., Guérin, R., Zhang, Z.L.: On the feasibility and efficacy of protection routing in ip networks. *IEEE/ACM Transactions on Networking* **19**(5), 1543–1556 (2011)
40. Larsen, K.G., Mariegaard, A., Schmid, S., Srba, J.: AllSynth: A BDD-based approach for network update synthesis. In: *Science of Computer Programming (SCICO)*. vol. 230 (2023)
41. Lee, C.Y.: Representation of switching circuits by binary-decision programs. *The Bell System Technical Journal* **38**(4), 985–999 (1959)
42. Liu, J., Panda, A., Singla, A., Godfrey, B., Schapira, M., Shenker, S.: Ensuring connectivity via data plane mechanisms. In: *NSDI* (2013)
43. Markopoulou, A., Iannaccone, G., Bhattacharyya, S., Chuah, C., Ganjali, Y., Diot, C.: Characterization of failures in an operational IP backbone network. *IEEE/ACM Trans. Netw.* **16**(4), 749–762 (2008)
44. Marnerides, A.K., Pezaros, D.P., Hutchison, D.: Internet traffic characterisation: Third-order statistics & higher-order spectra for precise traffic modelling. *Computer Networks* **134**, 183–201 (2018)
45. Miller, B.L., Goldberg, D.E., et al.: Genetic algorithms, tournament selection, and the effects of noise. *Complex systems* **9**(3), 193–212 (1995)
46. Pignolet, Y.A., Schmid, S., Tredan, G.: Load-optimal local fast rerouting for resilient networks. In: *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. pp. 345–356. IEEE (2017)
47. Prabhu, S., Chou, K.Y., Kheradmand, A., Godfrey, B., Caesar, M.: Plankton: Scalable network configuration verification through model checking. In: *17th USENIX Symposium on Networked Systems Design and Implementation ({NSDI} 20)*. pp. 953–967 (2020)
48. Reichert, C., Glickmann, Y., Magedanz, T.: Two routing algorithms for failure protection in ip networks. In: *10th IEEE Symposium on Computers and Communications (ISCC'05)*. pp. 97–102. IEEE (2005)
49. Schmid, S., Schnepf, N., Srba, J.: Resilient capacity-aware routing. In: *27th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. pp. 411–429. Springer (2021)
50. Schmid, S., Schou, M.K., Srba, J., Vanerio, J.: R-MPLS: Recursive protection for highly dependable MPLS networks. In: *Proc. ACM CoNEXT*. p. 276–292 (2022)
51. Schneider, T., Schmid, R., Vanbever, L.: On the complexity of network-wide configuration synthesis. In: *ICNP*. pp. 1–11. IEEE (2022)
52. Srinivas, M., Patnaik, L.M.: Genetic algorithms: A survey. *computer* **27**(6), 17–26 (1994)
53. Steffen, S., Gehr, T., Tsankov, P., Vanbever, L., Vechev, M.: Probabilistic verification of network configurations. In: *Proc. ACM SIGCOMM*. p. 750–764 (2020)
54. Stephens, B., Cox, A.L., Rixner, S.: Plinko: Building provably resilient forwarding tables. In: *HotTopics in Networks*. pp. 1–7 (2013)
55. Yang, B., Liu, J., Shenker, S., Li, J., Zheng, K.: Keep forwarding: Towards k-link failure resilient routing. In: *INFOCOM*. pp. 1617–1625 (2014)
56. Yu, P., Yang, Q., Fu, F., Kwak, K.S.: Inter-cell cooperation aided dynamic base station switching for energy efficient cellular networks. In: *2012 18th Asia-Pacific Conference on Communications (APCC)*. pp. 159–163 (2012)